SLIDE 1 / 46 · 20 secondes

Derrière les flags

À DIRE

« Ces archives racontent des challenges résolus pendant le CTF. Nous allons reprendre les raisonnements, un à un, avec des schémas et les preuves conservées. On commence par un cas très visuel : un texte transformé en nombres, puis retrouvé grâce aux tables publiques de GPT-2. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

QUESTIONS PROBABLES

SOURCES

Journaux des challenges, PowerPoint Tapo et fichiers fournis.

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

Embeddings — le défi

À DIRE

« On nous donne un fichier appelé Dockerfile.json. Malgré son nom, il contient du JSON et des nombres. Il indique le modèle GPT-2. Il y a 24 groupes de 768 nombres. Le flag a été transformé en cette représentation. Notre mission est de retrouver le texte qui se cache derrière. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le fichier contient session_id, model="gpt2" et inputs_embeds. La forme 24 × 768 est directement vérifiable dans le JSON. Un vecteur est simplement une liste de nombres. Un token est un morceau de texte : parfois un mot, parfois quelques lettres, parfois un signe.

QUESTIONS PROBABLES

Faut-il connaître le machine learning ? Non : il suffit de comprendre une table de correspondance et une addition. Le schéma explique la structure confirmée par la résolution, pas une hypothèse prise pour un fait au début.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:05, 20:09 · DOCKERFILE.JSON

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

Embeddings — le dictionnaire

À DIRE

« Un embedding, ici, c'est une liste de nombres associée à un morceau de texte. Imaginez un dictionnaire : à gauche, "BA" ; à droite, 768 nombres. GPT-2 possède cette table. Dans notre fichier, une deuxième liste de nombres a été ajoutée pour indiquer la place du morceau dans le texte. Et les deux tables sont disponibles dans le modèle public. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les tables sont wte, avec 50 257 lignes de tokens, et wpe, avec les positions. Dans ce challenge, vecteur[i] = wte[token_i] + wpe[i]. Les tokens BA, 26 et { correspondent effectivement aux trois premiers tokens retrouvés (4339, 2075, 90). Les barres sont une illustration, pas une projection calculée des 768 dimensions.

QUESTIONS PROBABLES

Un token est-il une lettre ? Pas forcément. Les trois premiers sont BA, 26 et {. Il ne faut donc pas annoncer 24 lettres : il y a 24 morceaux. Les tables ont été téléchargées pendant le CTF après autorisation ; rien n'a été téléchargé pour cette présentation.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/WRITEUP.MD, SECTIONS 2–4

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

SLIDE 4 / 46 · 45–60 secondes

Embeddings — une addition réversible

À DIRE

« Prenons trois nombres pour que le principe tienne sur un écran. Le token vaut deux, cinq, un. Sa position vaut un, zéro, trois. En additionnant colonne par colonne, on obtient trois, cinq, quatre. On connaît la position : on la retire et on retrouve deux, cinq, un. Il ne reste qu'à chercher cette ligne dans le dictionnaire. Dans le vrai fichier, c'est la même opération sur 768 nombres. »

DÉROULÉ DES ANIMATIONS

1. Montrer les deux listes à additionner.
2. → Les nombres reçus et la position connue.
3. → Retrouver la ligne de départ.
4. → Transposer aux 768 dimensions.

POUR COMPRENDRE

Exemple construit pour expliquer une addition de vecteurs. Ces valeurs ne représentent ni un token GPT-2 réel ni une ligne du fichier. La structure confirmée est E[i] = wte[token_i] + wpe[i], avec des écarts d'arrondi.

QUESTIONS PROBABLES

Pourquoi ne pas demander à un chatbot ? Il s'agit de retrouver des lignes dans les tables, pas de générer une phrase plausible.

SOURCES

ILLUSTRATION PÉDAGOGIQUE · STRUCTURE CONFIRMÉE DANS CLAUDE/WRITEUP.MD

LIMITES ET PROVENANCE

Les valeurs sont fictives ; la structure du calcul correspond au challenge.

Embeddings — le chemin inverse

À DIRE

« Prenons le premier groupe. Nous savons qu'il contient les nombres du morceau et ceux de sa position. On retire les nombres de position. Il reste presque exactement une ligne du dictionnaire. On cherche la ligne la plus proche : elle correspond à “BA”. Puis on refait la même chose pour le deuxième groupe, puis le troisième, jusqu'au bout. »

DÉROULÉ DES ANIMATIONS

1. Lire un groupe de nombres.
2. → Retirer sa composante de position.
3. → Chercher le token le plus proche.
4. → Répéter et assembler.

POUR COMPRENDRE

L'agent a chargé les poids publics de GPT-2, soustrait wpe pour les positions 0 à 23, puis comparé chaque vecteur aux 50 257 lignes de wte avec une distance euclidienne. Les indices retrouvés ont été passés au tokenizer pour reconstituer la chaîne.

QUESTIONS PROBABLES

Pourquoi « la plus proche » et pas « identique » ? Les nombres conservés présentent de très petits écarts. Après correction, les distances observées sont de 0,000 à 0,004. Le plus proche voisin suffit. Il n'y a pas de génération de texte ou de prompt envoyé à un chatbot.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/SRC/INVERT.PY · CLAUDE/OUT-01.TXT

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

SLIDE 6 / 46 · 45–60 secondes

Embeddings — vérifier plutôt que deviner

À DIRE

« L'agent compare d'abord les nombres directement à la table : le flag apparaît déjà. Mais les distances restent élevées. Après retrait de la position, les mêmes tokens sont retrouvés et les distances deviennent presque nulles. Cette correction explique les nombres reçus, en plus de produire un texte plausible. »

DÉROULÉ DES ANIMATIONS

1. Comparer les vecteurs bruts.
2. → Retirer la position : les distances chutent.
3. → Le texte retrouvé reste identique.
4. → Conclure sur les correspondances presque exactes.

POUR COMPRENDRE

La sortie conservée donne les distances euclidiennes 3,5 à 9,9 pour les vecteurs bruts et 0,000 à 0,004 après soustraction de wpe. Les cosinus passent de 0,35–0,74 à 1,000. Les bandes sont schématiques, sans échelle quantitative commune.

QUESTIONS PROBABLES

La première méthode avait-elle échoué ? Non : elle donnait déjà le bon texte. La correction confirme la structure.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/OUT-01.TXT · CLAUDE/WRITEUP.MD §4

LIMITES ET PROVENANCE

Mesures lues dans la sortie conservée. Aucune recherche relancée.

SLIDE 7 / 46 · 45–60 secondes

Embeddings — le résultat

À DIRE

« En assemblant les morceaux, on retrouve le flag. La sortie conservée le montre, et les mesures confirment des correspondances presque exactes. La leçon : ici, remplacer le texte par ses embeddings ne le rendait pas secret. On connaissait le modèle, ses tables étaient publiques, et l'opération pouvait être inversée. »

Clin d'œil facultatif à dire : « Il avait changé de tenue, mais laissé son nom dans le vestiaire. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le résultat brut, avant soustraction des positions, donnait déjà les bons tokens, avec des distances bien plus grandes. La correction rend la correspondance convaincante : distances de 0,000 à 0,004 et similarités cosinus affichées à 1,000. Le script final est claude/src/invert.py ; la sortie est claude/out-01.txt.

QUESTIONS PROBABLES

Est-ce vrai de tous les embeddings ? Cette démonstration porte sur ces embeddings d'entrée GPT-2, formés par une table de tokens et une table de positions. Il ne faut pas présenter cette recette comme une inversion universelle de tous les vecteurs de tous les modèles.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:09 · CLAUDE/OUT-01.TXT

LIMITES ET PROVENANCE

La capture haute résolution a été produite après l'événement à partir de la sortie sauvegardée. Le solve n'a pas été relancé pour cette présentation.

SLIDE 8 / 46 · 45–60 secondes

ID, please — deux portes d’entrée

À DIRE

« Le site propose une connexion avec un mot de passe, ou avec Google. Dans la connexion Google, on transmet au site une attestation qui indique quel compte se connecte. Cette attestation devrait être vérifiée. Or le chemin appelé “legacy” accepte même une attestation sans signature. On peut donc fabriquer ce qu’elle contient. Mais cela ne suffit pas encore : il faut désigner le bon compte. »

DÉROULÉ DES ANIMATIONS

1. Présenter les deux modes de connexion.
2. → Expliquer l’attestation Google en mots simples.
3. → Révéler l’absence de vérification de signature.

POUR COMPRENDRE

Les journaux décrivent POST /member/auth/socialLogin, qui accepte un JWT non signé avec alg:none et répond “legacy compatibility login”. JWT est le format de l’attestation. Le champ sub contient l’identifiant du compte Google. La session créée par l’application est ensuite un cookie opaque, distinct de ce JWT. Les détails du format ne sont pas nécessaires pour suivre la démonstration.

QUESTIONS PROBABLES

Pourquoi une signature ? Elle permet au site de contrôler l’origine et l’intégrité de l’attestation. Lire un numéro dans un document qu’un visiteur peut fabriquer ne prouve pas que ce visiteur possède le compte correspondant.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:25 · WRITEUP, SECTIONS 1–2

LIMITES ET PROVENANCE

Schéma pédagogique : aucune capture de connexion n’est présentée comme une preuve. L’analyse reprend les observations conservées ; aucune requête n’a été relancée.

SLIDE 9 / 46 · 45–60 secondes

ID, please — la mauvaise identité

À DIRE

« Les premières tentatives étaient intuitives : écrire “je suis admin” dans le document, ou mettre l’email de l’admin. Elles échouaient. Le rôle venait du compte déjà enregistré dans la base, et le site retrouvait ce compte avec un champ appelé sub. Même email, mais autre sub : autre compte, simple utilisateur. Le vrai problème devient donc : comment retrouver le numéro du compte admin ? »

Clin d’œil facultatif à dire : « Je me suis autoproclamé chef. Même le serveur a trouvé ça un peu rapide. »

DÉROULÉ DES ANIMATIONS

1. Question 1 : demander le rôle admin ne suffit pas.
2. → Question 2 : garder le même email ne retrouve pas le compte.
3. → Question 3 : garder le même sub retrouve le même compte.
4. → Conclure : il faut le numéro exact du compte admin.

POUR COMPRENDRE

Le comportement a été vérifié par plusieurs agents : même sub, même membre ; nouveau sub avec un email déjà connu, nouveau membre. Les champs role/isAdmin ne changent pas le rôle. L’enquête a aussi écarté de nombreuses pistes de mutation de comptes et d’injection, mais le public n’a pas besoin d’en mémoriser la liste. L’identifiant sub est la pièce qui relie l’attestation au membre stocké.

QUESTIONS PROBABLES

Pourquoi ne pas inventer un numéro ? Il désignerait généralement un nouveau compte utilisateur. Le résultat recherché est de retrouver la ligne existante de l’admin, pas de créer un compte qui porte un nom convaincant. Les essais de nombres ou mots courants n’ont pas retrouvé le bon compte.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:12, 20:25 · WRITEUP, SECTION 3

LIMITES ET PROVENANCE

Les résultats présentés sont ceux des essais documentés. Ils n’établissent pas qu’aucune autre attaque ne serait possible. Les identifiants de membres obtenus avant les remises à zéro pouvaient appartenir à d’autres participants.

SLIDE 10 / 46 · 45–60 secondes

ID, please — retrouver le numéro

À DIRE

« Un indice était sur la page d'accueil : le pied de page nommait Tech Corp Chandler et donnait une adresse Gmail. L'agent a utilisé une session Google de test pour chercher cet email via People API. La réponse lui a donné le numéro Google du compte, aussi appelé Gaia ID. Dans ce challenge, c'est le sub attendu. On peut maintenant fabriquer une attestation qui désigne le vrai compte admin. »

DÉROULÉ DES ANIMATIONS

1. Montrer l'indice de la page d'accueil.
2. → Décrire une recherche avec une session Google de test.
3. → Révéler l'identifiant abrégé.
4. → Résumer le changement d'approche.

POUR COMPRENDRE

Le journal de solve et le write-up documentent la recherche de techcorpchandler@gmail.com et le numéro 103465053501192984574. Le script claude/gaia_lookup.py conserve la logique de recherche. Aucun cookie, en-tête d'authentification ou secret de session n'est intégré à la présentation. Le numéro abrégé sur la slide est seulement un repère visuel.

QUESTIONS PROBABLES

Le numéro est-il un mot de passe ? Non : c'est un identifiant. La faille apparaît lorsque le site accepte ce numéro dans une attestation dont il ne vérifie pas l'origine. La présentation n'affirme pas qu'on peut résoudre n'importe quel email Google : elle décrit la recherche qui a fonctionné pour ce compte CTF.

SOURCES

ID_PLEASE · JOURNAL 20:35, 21:25 · WRITEUP, SECTIONS 4–5

LIMITES ET PROVENANCE

Le journal et le write-up documentent le résultat de la recherche. Le script est conservé, mais les cookies sont hors du dépôt. Un code HTTP 401 observé plus tôt ne prouve pas à lui seul qu'un compte existe ou qu'il est social ; le succès final confirme ici le compte ciblé.

SLIDE 11 / 46 · 45–60 secondes

ID, please — une identité usurpée

À DIRE

« Avec le vrai identifiant, l'attestation fabriquée retrouve le membre numéro 1, qui est administrateur. La session obtenue permet alors de lire le flag sur /api/flag. Le journal consigne le solve à 21 h 25. La leçon tient en une phrase : connaître le numéro d'un compte n'est pas une preuve qu'on en est le propriétaire. Le site devait vérifier l'attestation, et pas seulement lire le numéro qu'elle contenait. »

DÉROULÉ DES ANIMATIONS

1. Présenter la chaîne finale.
2. → Révéler les résultats consignés dans le write-up.
3. → Montrer le flag et la leçon.

POUR COMPRENDRE

Le payload final porte le vrai sub et le compte email. Le rôle n'est toujours pas ajouté par le token : il vient de la ligne administrateur déjà stockée. Le write-up transcrit socialLogin avec memberId=1 et role=admin, /api/me avec le membre admin, puis /api/flag avec le flag. Il corrige l'hypothèse précédente sur /api/admin/flag : cette route renvoie 404 même comme admin.

QUESTIONS PROBABLES

Qui a fait quoi ? Le write-up attribue la reconnaissance et les exclusions de pistes à plusieurs agents (pi, Codex, agy, qwen) et le déclic avec le solve final à Claude. Pourquoi tant d'impasses ? L'authentification était faible, mais il fallait d'abord trouver le sub exact ; les changements de rôles ou d'email ne désignaient pas le bon compte.

SOURCES

ID_PLEASE · JOURNAL 21:25 · WRITEUP, SECTIONS 6–8

LIMITES ET PROVENANCE

Preuve disponible : entrée de solve et réponse transcrite dans WRITEUP.md. Aucune capture ni sortie brute finale séparée n'a été trouvée dans les fichiers inspectés. Le panneau de cette slide est une transcription du write-up, pas une capture du service. Aucun exploit n'a été relancé.

Universal translator — retrouver un dictionnaire

À DIRE

« On reçoit un programme destiné à une machine inconnue. Il est composé de petits codes, mais on ne sait plus quelle action chaque code représente. La machine travaille avec une pile : elle dépose des valeurs, les copie, les déplace ou fait des calculs. Heureusement, on nous donne aussi quatre exemples et les résultats qu'ils doivent produire. Ces exemples vont servir à reconstruire le dictionnaire. »

Clin d'œil facultatif à dire : « Elle savait très bien ce qu'elle disait. Le problème, c'était notre niveau en machine à pile. »

DÉROULÉ DES ANIMATIONS

1. Montrer les codes dont le sens manque.
2. → Expliquer la pile en une phrase.
3. → Montrer les quatre exemples : ils servent de tests.

POUR COMPRENDRE

L'archive contient deux README, quatre échantillons binaires et transmission.bin, un programme de 176 octets. Les codes 0x00 à 0x07 sont une permutation de huit opérations de pile ; les codes 0x10 à 0x19 sont une permutation de dix opérations arithmétiques. Les autres octets ajoutent directement une valeur à la pile. Une pile se lit depuis son sommet, la dernière valeur ajoutée. Les cases affichées sont un schéma, pas un état extrait des échantillons.

QUESTIONS PROBABLES

Pourquoi ne pas essayer les instructions au hasard ? Il y a un ensemble fini de dictionnaires possibles. Les réponses connues permettent de rejeter automatiquement ceux qui ne conviennent pas. Ici, le défi est de retrouver le langage de la machine, puis d'interpréter la transmission.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:48–18:51 · README DE L'ARCHIVE

LIMITES ET PROVENANCE

challenge.md est un modèle vide. Le journal conserve le texte fourni par l'utilisateur et les règles lues dans le README.

Universal translator — voir la pile et le test

À DIRE

« Prenons un tout petit exemple, inventé pour comprendre. On a empilé 7, puis 3 : le 3 est donc au sommet. On cherche le sens d'une instruction, avant d'afficher le sommet. Si elle copie, on obtient 3, 3, 7 : la sortie est 3. Si elle échange les deux valeurs, le 7 passe au sommet : la sortie est 7. Si l'exemple nous dit que la sortie doit être 3, on peut éliminer "échanger". C'est ce principe que l'agent automatise sur les vrais programmes. »

DÉROULÉ DES ANIMATIONS

1. Lire la pile depuis le dessus : 3, puis 7.
2. → Si l'instruction copie, la sortie serait 3.
3. → Si elle échange, la sortie serait 7.
4. → Révéler la réponse attendue 3 : on élimine l'échange.

POUR COMPRENDRE

Le sommet est dessiné en haut. Copier duplique la dernière valeur ; échanger permute les deux dernières. Aucun code hexadécimal réel n'est associé à l'instruction inconnue dans ce mini-exemple. La pile et la sortie attendue sont choisies pour enseigner la mécanique, pas extraites des échantillons du challenge. Dans le mapping réellement trouvé, DUP vaut 0x03 et SWAP vaut 0x06.

QUESTIONS PROBABLES

Ce test suffit-il à trouver toute la machine ? Non. L'hypothèse compatible a seulement passé un test. D'autres opérations pourraient produire la même sortie dans un cas particulier. Le solve utilise tous les exemples fournis pour départager les dictionnaires complets.

SOURCES

SCHÉMA PÉDAGOGIQUE · RÈGLES DE PILE DÉCRITES DANS LE README

LIMITES ET PROVENANCE

Les nombres 7 et 3 et la sortie attendue 3 sont un exemple pédagogique inventé, explicitement signalé sur la slide. Ce n'est pas une trace du solve.

Universal translator — les vrais tests

À DIRE

« Revenons au vrai challenge. L'agent commence par les actions sur la pile : copier, échanger, supprimer, émettre… Les deux premiers exemples permettent de retrouver leur correspondance avec les codes. Puis il cherche les opérations de calcul. Un exemple laisse encore 346 possibilités dans la bonne interprétation des nombres. Le dernier n'en laisse plus qu'une. Le dictionnaire peut enfin servir à lire la transmission inconnue. »

DÉROULÉ DES ANIMATIONS

1. Deux groupes d'instructions sont résolus successivement.
2. → Les exemples 1 et 2 fixent les actions de pile.
3. → Les exemples 3 et 4 fixent les calculs : 346 puis 1.
4. → Le dictionnaire confirmé sert à lire la transmission.

POUR COMPRENDRE

Le solve est une recherche exhaustive. stack_candidates.py teste les 8! permutations des opérations de pile contre les exemples 01 et 02. arithmetic_candidates.c teste ensuite les 10! permutations arithmétiques. L'exemple 03 laisse 428 candidats non signés ou 346 signés. Après l'exemple 04, il reste zéro candidat non signé et un seul signé. Le modèle correct utilise des entiers signés sur 64 bits pour MAX, MIN et le décalage à droite. Z3 était installé dans l'environnement, mais les solveurs conservés font de l'énumération exhaustive.

QUESTIONS PROBABLES

Pourquoi préciser les nombres signés ? Une valeur peut représenter un nombre négatif. Cela change les comparaisons et certains décalages. L'agent a testé les deux conventions ; les sorties connues ont confirmé laquelle convenait. Ne pas présenter 346 comme le nombre total de toutes les possibilités initiales.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:51–18:53 · CODEX/SOLVER-OUTPUT.TXT

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

Universal translator — la transmission lue

À DIRE

« Le dictionnaire est maintenant retrouvé. Par exemple, 03 signifie copier le sommet de la pile, 04 signifie émettre une valeur, et 19 signifie additionner. L'agent utilise cette machine reconstruite pour lire transmission.bin. Elle produit des octets qui, convertis en texte, donnent le flag. La preuve est la sortie conservée du solveur. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le mapping complet de pile est 00=ROT, 01=TUCK, 02=OVER, 03=DUP, 04=EMIT, 05=NIP, 06=SWAP, 07=DROP. Le mapping arithmétique est 10=RSHIFT, 11=XOR, 12=LSHIFT, 13=*, 14=AND, 15=MIN, 16=MAX, 17=-, 18=OR, 19=+. Les codes sont en hexadécimal. La transmission produit les octets 42 41 32 36 7b… qui deviennent BA26{0n3_st4ck_f0rth_4ll} en ASCII. Les scripts finaux sont codex/stack_candidates.py, codex/generate_data.py et codex/arithmetic_candidates.c.

QUESTIONS PROBABLES

L'agent a-t-il contacté un serveur ? Aucun serveur cible n'est spécifié dans les archives de ce challenge. Le travail documenté est local, sur les fichiers fournis. La présentation ne relance pas le solveur.

SOURCES

UNIVERSAL_TRANSLATOR · CODEX/SOLVER-OUTPUT.TXT · JOURNAL 18:53

LIMITES ET PROVENANCE

La capture a été annotée après l'événement. La correspondance et les octets décodés sont aussi consignés dans la sortie texte conservée.

Silent beacon — le programme et le leurre

À DIRE

« On reçoit une copie de la mémoire d'une balise. Son programme demande un code d'activation de huit caractères hexadécimaux. Si le code est correct, il décode un enregistrement. En cherchant les textes lisibles dans le fichier, on tombe aussi sur un flag tout prêt. C'est un leurre. L'objectif est de comprendre le contrôle du code, puis de suivre le vrai décodage. »

Clin d'œil facultatif à dire : « Il ne manquait que le panneau : absolument pas un piège. »

DÉROULÉ DES ANIMATIONS

1. Présenter le dump de mémoire.
2. → Présenter le contrôle du code.
3. → Montrer le leurre en clair.
4. → Dire ce qui fera réellement preuve.

POUR COMPRENDRE

Le dump beacon_flash.bin est identifié comme un firmware ESP32-C6. Le contrôle d'activation et le décodage sont dans le programme fourni. Le faux flag BA26{n0t_th15_0n3_k33p_d1gg1ng} est visible en clair à l'offset 0x19d00. La reconstruction montre que le programme en lit seulement le premier caractère pour un message de démarrage ; le chemin qui affiche UNSEALED utilise une autre zone de données.

QUESTIONS PROBABLES

Le faux flag a-t-il été soumis ? Les archives ne le disent pas. Ne pas le prétendre. Avions-nous l'appareil physique ? Les sources documentent un dump et un solveur hors ligne, pas une activation sur un appareil.

SOURCES

SILENT_BEACON · JOURNAL 18:45, 18:49 · BACKFILL 20:57

LIMITES ET PROVENANCE

Le texte complet du challenge, les commandes exactes de désassemblage et une éventuelle activation réelle ne sont pas conservés. Le schéma mémoire est pédagogique, pas une carte à l'échelle.

Silent beacon — le contrôle d’activation

À DIRE

« Le contrôle est entièrement dans le programme. Il prend deux textes lisibles dans le dump et calcule deux nombres, des empreintes. Il les combine ensuite suivant une recette fixe. Le résultat est B80FEF82. C’est à ce nombre qu’il compare le code fourni. L’agent n’a pas essayé tous les codes : il a refait le calcul qu’il venait de lire. »

DÉROULÉ DES ANIMATIONS

1. Les ingrédients du calcul sont les deux textes du dump.
2. → Le programme les transforme en deux empreintes.
3. → Il les combine pour obtenir le code attendu.
4. → Le contrôle compare le code fourni à cette réponse.

POUR COMPRENDRE

Les textes sont LilyGO T7-C6 et BA26-BEACON-01. Le contrôle applique FNV-1a 32 bits à chacun. Les hashes conservés sont 56ea3ff6 pour LilyGO T7-C6 et cd1014a9 pour BA26-BEACON-01. Le code attendu est rotl32(56ea3ff6,7) XOR cd1014a9 = B80FEF82. Ces valeurs viennent de la sortie sauvegardée, pas d’un recalcul pour la présentation. L’opération de rotation déplace les bits en boucle ; XOR combine les bits. Les deux termes techniques restent facultatifs à l’oral.

QUESTIONS PROBABLES

Qu’est-ce qu’une empreinte ? Ici, un nombre obtenu de façon déterministe à partir d’un texte. Le point important n’est pas de retenir FNV, mais que le texte et la recette sont connus. Une tentative initiale a inversé les deux hashes et donné DEE06B10 : leur ordre est donc essentiel.

SOURCES

SILENT_BEACON · CODEX/SOLVE.PY · CLAUDE/OUT-03-SOLVE.TXT

LIMITES ET PROVENANCE

La comparaison affichée est un schéma de la logique analysée. Les archives ne prouvent pas qu’un code ait été saisi sur un appareil ou un service. L’erreur initiale est consignée, mais sa sortie n’a pas été conservée.

Silent beacon — deux calculs distincts

À DIRE

« Il faut bien distinguer deux choses. Le premier calcul sert à vérifier le code d'activation. Le second utilise les mêmes empreintes pour générer une suite de nombres. On combine cette suite avec les octets chiffrés du dump, un octet à la fois, et le texte devient lisible. Le code attendu n'est donc pas simplement une clé qu'on applique au fichier : le solveur reproduit aussi cette deuxième recette. »

DÉROULÉ DES ANIMATIONS

1. Les deux empreintes servent au contrôle déjà expliqué.
2. → Elles servent aussi à produire une suite pour décoder.
3. → Combiner cette suite avec les 31 octets permet de lire le texte.
4. → Le solveur reproduit les deux étapes.

POUR COMPRENDRE

Le décodeur utilise les deux hashes comme un état initial 64 bits de xorshift, avec les décalages 13, 7 et 17. Le hash de LilyGO T7-C6 est le mot haut ; celui de BA26-BEACON-01 est le mot bas. À chaque étape, le programme produit un nouvel état et combine son octet bas par XOR avec un octet des 31 octets stockés à l'adresse 0x42039d40. La représentation par des points est symbolique : elle n'affiche pas des valeurs de flux calculées.

QUESTIONS PROBABLES

Pourquoi le bon code ne suffit-il pas à expliquer le flag ? Il explique le contrôle d'accès, mais il faut aussi savoir comment le contenu est transformé pour justifier le texte retrouvé. Ici, toutes les données et opérations nécessaires sont présentes dans le dump ; le solveur peut donc les reproduire hors ligne.

SOURCES

SILENT_BEACON · CODEX/SOLVE.PY · JOURNAL 18:49, BACKFILL 20:57

LIMITES ET PROVENANCE

Le schéma distingue deux usages réels des hashes ; il n'est pas une capture d'exécution. Aucun décodeur n'a été relancé pour préparer cette slide.

Silent beacon — la preuve du décodage

À DIRE

« Le solveur retrouve le code attendu et décode l'enregistrement. Cette fois, le flag vient bien du chemin de succès du programme, et il est enregistré dans la sortie du solveur. On sait donc expliquer comment il a été obtenu. C'est la différence avec le faux flag trouvé en clair : l'analyse relie le résultat au mécanisme réel du challenge. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

codex/solve.py lit le dump, récupère les textes d'identité, calcule les hashes et le code, puis refait le xorshift et les XOR sur les 31 octets. La sortie conservée indique activation code=B80FEF82 et record=BA26{x0rsh1ft_s34ls_th3_b34c0n}. Les étapes techniques détaillées sont dans le journal original de Codex et dans la reconstruction à partir du désassemblage conservé.

QUESTIONS PROBABLES

Est-ce une preuve de déverrouillage matériel ? Non : la preuve documentée est le calcul et le décodage hors ligne. Le journal ne conserve aucune trace d'une saisie du code sur un appareil ou un service. Ce que l'on peut affirmer est que le solveur reproduit la logique analysée et retrouve le résultat journalisé.

SOURCES

SILENT_BEACON · CLAUDE/OUT-03-SOLVE.TXT · JOURNAL 18:49

LIMITES ET PROVENANCE

La capture de sortie a été produite lors du backfill à partir d'une reproduction hors ligne. Elle concorde avec le code d'activation et le flag consignés par Codex pendant le solve.

Wrong path — sortir du dossier

À DIRE

« FileVault nous laisse déposer puis lire des fichiers. Le serveur doit rester dans le dossier de notre compte. Regardons la requête documentée : on demande “deux fois point point slash, challenge.db”. Le serveur l’ajoute au chemin de notre compte. Le premier “../” remonte de notre compte vers uploads ; le second remonte vers app. La cible devient donc /app/challenge.db : la base de comptes. Une autre route, /download, bloquait bien cette sortie. L’agent a utilisé /raw. »

Clin d’œil facultatif à dire : « On avait accès à notre tiroir. Deux points et un slash plus tard, on visitait le meuble. »

DÉROULÉ DES ANIMATIONS

1. Lire la requête et nommer le dossier autorisé.
2. → Montrer le chemin construit et le sens de ../.
3. → Montrer le fichier final hors du dossier.
4. → Comparer la route sûre et la route vulnérable.

POUR COMPRENDRE

La requête GET /raw?filename=../../challenge.db et la racine /app/uploads/<username>/ figurent dans JOURNAL.md, sections App et Exploitation. Le journal décrit open(os.path.join(UPLOAD_FOLDER,user,filename)) sans assainissement pour /raw et /view, contre send_from_directory pour /download. La résolution présentée illustre la sémantique de ../ ; elle n’est pas une réponse HTTP récupérée à nouveau. Avant la base, la lecture de ../../app.py avait confirmé son emplacement. /etc/passwd a servi de preuve de traversée. L’essai ..../// a échoué ; ../ et sa forme encodée ont fonctionné selon les notes.

QUESTIONS PROBABLES

Pourquoi os.path.join ne protège-t-il pas ? Il compose un chemin ; le mécanisme documenté n’impose pas ensuite que le fichier final reste sous le dossier autorisé. On ne peut pas déduire de ce solve que toutes les routes de l’application étaient vulnérables : le contraste /download versus /raw est essentiel.

SOURCES

WRONG_PATH/JOURNAL.MD · APP, EXPLOITATION, SOLVED 20:07

LIMITES ET PROVENANCE

Aucune capture de l’interface web ni requête/réponse brute n’a été conservée. Les chemins et la requête sont transcrits du journal ; le nom de compte est remplacé par un repère générique.

Wrong path — le faux flag

À DIRE

« La base contient un mot de passe qui ressemble à un flag. L'agent pense d'abord avoir fini, puis corrige cette conclusion : c'était un piège. Les vrais identifiants administrateur sont dans la même base. Une fois connecté comme admin, il voit un fichier qui indique que le vrai flag a été caché dans un labyrinthe. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le journal contient une correction explicite : la chaîne de type BA26-dftwp-… est un leurre, pas le résultat final. La lecture de /proc/self/environ a révélé SECRET_KEY et confirmé la localisation des identifiants. Le compte admin de challenge.db a permis l'accès à /dashboard, où figuraient flag.txt et flag.maze. Le solve final utilise les identifiants ; il ne faut pas inventer une falsification de session avec SECRET_KEY.

QUESTIONS PROBABLES

Comment sait-on que le premier flag était faux ? Le journal le corrige explicitement et le résultat réel est conservé dans claude/out-01.txt. La présentation suit cette correction, pas le premier résumé obsolète.

SOURCES

WRONG_PATH · CLAUDE · JOURNAL, CORRECTION ET RECONSTRUCTION

LIMITES ET PROVENANCE

Le journal reconstitue l'ordre général des étapes après l'événement. Il n'existe pas de traces HTTP permettant de dater chaque action précisément.

Wrong path — comprendre BFS

À DIRE

« Le labyrinthe comporte plus de quarante mille cases : le lire à l'œil serait laborieux. Le programme utilise une recherche en largeur. Il garde une file de cases à explorer. Quand il visite une case, il ajoute ses voisins accessibles qu'il n'a pas encore vus, en mémorisant d'où ils viennent. Dès que la sortie est trouvée, il remonte ces liens pour reconstruire le chemin. Seules les lettres sur ce chemin seront gardées. »

DÉROULÉ DES ANIMATIONS

1. Lire les cinq lignes de l'algorithme sans détailler Python.
2. → Expliquer pourquoi chaque case garde un prédécesseur.
3. → Revenir de la sortie vers l'entrée.
4. → Garder les lettres de la route, pas celles des branches.

POUR COMPRENDRE

claude/solve.py lit la grille, repère les deux caractères >, puis initialise prev={start:None} et q=deque([start]). Les quatre voisins sont testés ; une case est admissible si elle reste dans la grille et n'est pas un #. Le dictionnaire prev sert à la fois de marqueur de visite et de lien vers le prédécesseur. Le solveur s'arrête en atteignant la sortie, reconstruit puis renverse le chemin, et retire les murs, espaces et portes avant de joindre les lettres. Le pseudocode de la slide simplifie exactement ces opérations.

QUESTIONS PROBABLES

Pourquoi conserver le prédécesseur ? Une liste de cases visitées ne donne pas directement la route finale : elle inclut aussi des branches inutiles. Les liens permettent de repartir de la sortie et de retrouver la suite exacte des cases. Aucun besoin d'expliquer des graphes formels pour suivre cette idée.

SOURCES

WRONG_PATH/CLAUDE/SOLVE.PY · FILE DE CASES, PREV, RECONSTRUCTION

LIMITES ET PROVENANCE

Le pseudocode est une traduction pédagogique du solveur conservé, pas une trace de son exécution. La recherche n'a pas été relancée pour produire la présentation.

Wrong path — le labyrinthe

À DIRE

« Le fichier est un labyrinthe en texte : des murs, des couloirs et des lettres. Le solveur explore les passages depuis l'entrée jusqu'à la sortie. Il conserve ce chemin, puis lit les caractères qui s'y trouvent. Ensemble, ils forment le vrai flag. Le dessin vert est un rendu du chemin retrouvé à partir du fichier conservé. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le labyrinthe mesure 201 × 201. Les deux portes sont en (ligne 1, colonne 0) et (ligne 199, colonne 200). claude/solve.py utilise une recherche en largeur, ou BFS, puis remonte les prédécesseurs pour reconstruire le chemin. La sortie sauvegardée indique une longueur de 2 783 positions. Le flag complet est BA26{tHis_Is_a_rEally_loNG_flag_ThAt_is_Very_DiFFicult_to_gUess_bUT_you_cAn_find_It_in_tHE_mAze}.

QUESTIONS PROBABLES

Qu'est-ce qu'une recherche en largeur ? Elle visite d'abord les cases voisines, puis leurs voisines, couche par couche. Le public peut retenir « explorer les couloirs en conservant d'où l'on vient ». Le chemin vert représente le résultat conservé, pas une animation de l'exécution d'origine.

SOURCES

WRONG_PATH/CLAUDE/SOLVE.PY · OUT-01.TXT · RENDER_MAZE.PY

LIMITES ET PROVENANCE

Le dessin a été créé après l'événement à partir du labyrinthe local. La chaîne sur la slide est volontairement abrégée ; la preuve et les notes donnent le résultat complet.

Peak privilege — le défi

À DIRE

« Ce système décide des droits à partir d'une image de badge. Il classe chaque image comme employé, invalide ou administrateur. Pour obtenir le flag, il faut que son score ADMIN atteigne 99,5 %. On ne possède pas le modèle, mais le service donne ses scores. C'est cette information qui va guider les modifications de l'image. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

L'entrée est une image RGB 256 × 176. Le traitement connu découpe un carré 96 × 96, le passe en niveaux de gris puis moyenne des blocs 3 × 3 pour obtenir une grille 32 × 32. L'agent peut donc contrôler 1 024 valeurs via des blocs de pixels. Les poids du modèle ne sont pas fournis. Le badge employé initial a un score EMPLOYEE proche de 1 et ADMIN d'environ $2{,}3 \times 10^{-23}$.

QUESTIONS PROBABLES

Est-ce une reconnaissance de carte d'identité ? Les archives décrivent un scanner artificiel NeuralBadge de CTF avec trois classes. Ne pas extrapoler ce résultat à un produit réel. Le score renvoyé est une sortie du modèle, pas une garantie de probabilité réelle.

SOURCES

PEAK_PRIVILEGE/JOURNAL.MD · SETUP · CLAUDE/EXTRACTED/PLAYER/PREPROCESS.PY

LIMITES ET PROVENANCE

La description vient du journal, car challenge.md est un modèle vide. L'image employé est un fichier fourni ; le scanner dessiné est une illustration.

Peak privilege — contrôler l’entrée

À DIRE

« Avant de chercher un badge gagnant, l’agent lit le traitement appliqué à l’image. Le modèle ne voit pas toute la carte : seulement ce carré. Il est converti en gris, puis chaque bloc de trois pixels sur trois devient une valeur moyenne. Cela donne une grille de 32 sur 32. Si on met neuf pixels identiques dans un bloc, on choisit exactement la valeur de la case correspondante. L’agent peut donc piloter toutes les entrées du modèle. »

DÉROULÉ DES ANIMATIONS

1. Pointer le carré lu dans le badge.
2. → Montrer ce qui est recadré puis moyenné.
3. → Expliquer comment neuf pixels identiques contrôlent une case.
4. → Conclure : toute l’entrée utile est contrôlable.

POUR COMPRENDRE

preprocess.py donne BADGE_SIZE=(256,176), PATTERN_BOX=(32,48,128,144) et une conversion de luminance 0,299R+0,587G+0,114B, suivie de reshape(32,3,32,3).mean(axis=(1,3)). Régler R=G=B=k sur un bloc donne k/255, soit 256 niveaux contrôlables par case. claude/work/badge.py fait l’opération inverse pour produire un PNG : grid_to_png agrandit la grille et la place dans le carré. Le journal rapporte un écart maximum de $1{,}2\times10^{-7}$ lors du test aller-retour. Le cadre et le recadrage montrés utilisent les coordonnées exactes de la spécification ; la grille de droite est schématique.

QUESTIONS PROBABLES

Pourquoi cette étape compte-t-elle ? Modifier du texte ou des pixels hors du carré serait inutile : ils sont éliminés avant le modèle. Les bonnes dimensions d’image ne suffisent pas ; il faut comprendre ce qui est effectivement transmis au classifieur. Le contrôle est quantifié sur 256 niveaux, pas sur des réels de précision infinie.

SOURCES

PEAK_PRIVILEGE/CLAUDE/EXTRACTED/PLAYER/PREPROCESS.PY · CLAUDE/WORK/BADGE.PY

LIMITES ET PROVENANCE

L’image est le badge fourni. Le recadrage et le cadre sont des explications visuelles du code fourni ; la grille 32 × 32 est représentée schématiquement, sans prétendre afficher ses 1 024 valeurs mesurées.

Peak privilege — plus chaud, plus froid

À DIRE

« L'agent part du vrai badge employé, car du bruit aléatoire donnait surtout INVALID. À chaque tour, il choisit vingt variations. Pour chacune, il teste le badge modifié dans un sens et dans le sens contraire : quarante images au total. Si un sens rapproche davantage de ADMIN, la différence des scores indique une direction utile. Il combine les vingt indications, modifie le badge et recommence. C'est "plus chaud, plus froid", mais mesuré et automatisé. »

DÉROULÉ DES ANIMATIONS

1. Expliquer le départ depuis un badge déjà reconnu.
2. → Comparer les deux sens d'une même variation.
3. → Combiner les directions et mettre à jour le badge.
4. → Répéter la boucle jusqu'à l'acceptation.

POUR COMPRENDRE

Le solveur nes.py construit x+sigma·eps[k] et x−sigma·eps[k] pour chacune des 20 perturbations gaussiennes. Son objectif est log(pADMIN) − log(max(pEMPLOYEE,pINVALID)). La différence des deux objectifs pondère la perturbation ; leur moyenne estime un gradient. Il normalise ce gradient, applique un pas de taille lr, puis borne x dans [0,1]. La conversion en PNG quantifie à 256 niveaux. Le meilleur score ADMIN et sa grille sont sauvegardés. Le journal rapporte deux phases : sigma=0,06 et lr=0,08, puis lr=0,28. Le script vérifie aussi le badge courant hors du lot de variations.

QUESTIONS PROBABLES

Choisit-il simplement le meilleur badge du lot ? Non : il estime une direction à partir des paires, puis construit une nouvelle image. Les vingt paires aident à agréger des signaux bruités. « Plus ADMIN » simplifie la comparaison : l'objectif tient également compte de la classe concurrente la plus forte. Toutes les étapes ne sont pas nécessairement meilleures que la précédente.

SOURCES

PEAK_PRIVILEGE/CLAUDE/WORK/NES.PY · SCORE, PAIRES ±EPS, GRADIENT, MISE À JOUR

LIMITES ET PROVENANCE

Les deux variantes sont représentées par leur rôle, sans image ni score inventé. Le journal conserve un résumé des essais, pas la série complète des retours HTTP.

SLIDE 27 / 46 · 45–60 secondes

Peak privilege — le badge accepté

À DIRE

« Le badge final est visuellement étrange. Pourtant, le journal rapporte un score ADMIN de 99,6 %, au-dessus du seuil de 99,5 %, puis l'accès accordé et le flag. L'image gagnante a été conservée. La leçon de ce challenge est que le modèle donne à l'attaquant une mesure qui l'aide à fabriquer une entrée acceptée. »

Clin d'œil facultatif à dire : « Pour la photo du badge, le modèle a clairement des critères différents des ressources humaines. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le journal rapporte environ 113 appels par lots de 40 images pour l'ensemble de la résolution ; ces nombres restent approximatifs. La seconde phase atteint le seuil après environ 53 itérations. Le badge est claude/work/admin_badge.png, avec la grille gagnante best_grid.npy. Le solveur final est nes.py. Le résultat et le flag sont consignés dans la section SOLVED.

QUESTIONS PROBABLES

La capture prouve-t-elle à elle seule le score ? Non : elle montre le badge gagnant. Le score, l'accès accordé et le flag viennent du journal. Aucun log brut des réponses du service n'est gardé ; la présentation le dit explicitement dans la provenance.

SOURCES

PEAK_PRIVILEGE/JOURNAL.MD · SOLVED · CLAUDE/WORK/ADMIN_BADGE.PNG, BEST_GRID.NPY

LIMITES ET PROVENANCE

Les notes originales sont non datées ; les horaires de la reconstruction viennent des dates de fichiers. Le score de 0,996 est rapporté dans le journal, sans réponse HTTP brute conservée.

Rucher — les deux fichiers

À DIRE

« On dispose de dix secondes de signal radio et du programme d'une balance. La résolution comporte trois couches. D'abord, transformer les ondes en octets : c'est du décodage radio. Ensuite, retrouver la clé Zigbee pour lire les rapports : c'est le chiffrement du réseau. Enfin, ces rapports contiennent des fragments encore chiffrés par l'application. Le firmware nous explique cette dernière règle, liée à l'identité de la balance. »

Clin d'œil facultatif à dire : « Pour un challenge nommé Rucher, le vrai buzz venait de la balance. »

DÉROULÉ DES ANIMATIONS

1. Distinguer le signal radio du programme embarqué.
2. → Montrer le chiffrement du réseau comme une deuxième couche.
3. → Montrer la protection propre à l'application.
4. → Annoncer que chaque étape a son propre test de vérification.

POUR COMPRENDRE

La capture SigMF de 80 Mo contient des échantillons ci8 à 4 MS/s, centrés sur 2 475 MHz. Le journal identifie IEEE 802.15.4 canal 25 et Zigbee. Le dump balance_flash.bin est un firmware ESP32-C6 partiel. Ses chaînes nomment Zigbee et la clé par défaut ; sa zone NVS est vierge, donc il ne contient pas la clé réseau provisionnée de la balance observée. Le journal attribue l'ensemble du solve à Claude. Les trois couches de la slide regroupent la démodulation, la sécurité Zigbee et la protection applicative RC4 ; elles ne prétendent pas représenter toutes les sous-couches officielles du protocole.

QUESTIONS PROBABLES

Pourquoi les deux fichiers sont-ils nécessaires ? La radio apporte les échanges d'appairage, les fragments et l'identité de l'appareil. Le firmware permet de comprendre le protocole et la fabrication de la dernière clé. Il n'y a pas de flag prêt à être lu dans ce dump de fabrication.

SOURCES

RUCHER/JOURNAL.MD · SECTIONS 0–4 · RUCHER.SIGMF-META · BALANCE_FLASH.BIN

LIMITES ET PROVENANCE

Le diagramme est une vue pédagogique des étapes du solve. La chronologie a été reconstruite à partir des fichiers ; le dump est tronqué et ne contient pas des partitions complètes hors de sa taille.

Rucher — des ondes aux paquets

À DIRE

« Au début, on n'a que des ondes enregistrées. L'agent reconnaît le protocole Zigbee et écrit un décodeur. Il compare le signal aux motifs attendus pour retrouver les nombres envoyés. Les paquets possèdent un contrôle d'erreur : c'est un bon moyen de vérifier le décodage. Résultat : les 42 paquets passent ce contrôle. On peut maintenant analyser les messages. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les scripts decode154.py et extract_all.py détectent les salves et emploient un filtre adapté complexe aux motifs O-QPSK/DSSS. Le suréchantillonnage ×4 améliore la précision. Le FCS CRC-16 sert à confirmer les conventions de modulation et les octets. Le résultat est enregistré dans claude/rucher.pcap, avec 42 salves décodées et des FCS valides. La capture comprend l'association puis des rapports périodiques.

QUESTIONS PROBABLES

Le contrôle d'erreur déchiffre-t-il le message ? Non. Il confirme que les bits radio ont été lus correctement. Le contenu peut encore être chiffré. C'est pour cela que le décodage et le déchiffrement sont deux étapes distinctes.

SOURCES

RUCHER/CLAUDE/DECODE154.PY · EXTRACT_ALL.PY · JOURNAL.MD, SECTION 2

LIMITES ET PROVENANCE

L'onde dessinée est une illustration du signal, pas une courbe mesurée. La quantité 42/42 vient du journal et du fichier de paquets conservé.

Rucher — la clé réseau

À DIRE

« La balance reçoit sa clé réseau quand elle rejoint la passerelle. L'échange est dans notre capture. La clé n'est pas envoyée en clair : le message est protégé par une clé de transport, dérivée d'une clé par défaut connue. L'agent applique cette dérivation, ouvre le message Transport-Key et vérifie son contrôle d'authenticité. Il dispose alors de la clé réseau. Elle ouvre les rapports et révèle six fragments, qui ont encore une protection applicative. »

DÉROULÉ DES ANIMATIONS

1. La distribution de clé a été enregistrée.
2. → Dériver la clé de transport depuis la clé par défaut.
3. → Ouvrir le message et vérifier son authenticité.
4. → Utiliser la clé réseau pour lire les six fragments.

POUR COMPRENDRE

Le message APS Transport-Key est protégé avec une clé de transport dérivée de ZigBeeAlliance09 par le hachage Zigbee AES-MMO avec l'octet 0x00. Il ne faut pas dire que la clé par défaut est directement appliquée au paquet sans cette dérivation. AES-CCM* permet de déchiffrer et de vérifier le MIC. La clé réseau récupérée est b8d8e46f82f99446568d2d30e9512c16. Les rapports NWK vérifiés contiennent six fragments indexés de 0 à 5, totalisant 46 octets.

QUESTIONS PROBABLES

La clé est-elle envoyée en clair ? Non : elle est chiffrée, mais sa protection dérive d'une clé par défaut connue. Pourquoi faut-il capturer l'appairage ? Dans ce solve, c'est l'échange conservé qui fournit la clé réseau nécessaire pour lire les rapports.

SOURCES

RUCHER/CLAUDE/SOLVE_FINAL.PY · ZB_KEYED_HASH, CCM_STAR, TRANSPORT-KEY

LIMITES ET PROVENANCE

Les algorithmes de dérivation et de chiffrement sont nommés dans les notes. La slide expose la dépendance entre les clés sans prétendre qu'une clé en clair circulait sur la radio.

Rucher — la couche liée à l’appareil

À DIRE

« Le firmware révèle comment est fabriquée la dernière clé. Elle concatène l’identifiant radio de la balance, le mot “propolis” et le numéro du fragment. L’identifiant, ici une adresse EUI-64 de huit octets, est visible dans la capture. Il fait que deux balances n’utilisent pas la même clé, mais il ne constitue pas un secret pour l’observateur. L’agent reconstruit donc chaque clé, applique RC4 normalement et assemble les six morceaux. Il ne s’agit pas d’une cassure mathématique de RC4. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

L’algorithme reconnu est RC4, réinitialisé pour chaque fragment. La clé exacte est EUI-64 en ordre little-endian, suivie des octets de “propolis”, puis d’un octet d’index. L’EUI-64 de la balance est 40:4c:ca:ff:fe:46:87:c4, soit c48746feffca4c40 en little-endian. Le solveur final claude/solve_final.py prend le pcap, récupère la clé réseau, déchiffre les rapports et la couche RC4, puis concatène les fragments. La sortie conservée confirme le flag affiché.

QUESTIONS PROBABLES

L’identité de la balance est-elle un secret ? Elle est observable dans la capture. Ici, elle personnalise le chiffrement, mais elle ne suffit pas à garder le contenu secret une fois le mécanisme compris. Pourquoi le firmware seul ne donne-t-il pas le flag ? Il manque les messages de cette balance et son identité observée.

SOURCES

RUCHER/CLAUDE/SOLVE_FINAL.PY · CLÉ PAR FRAGMENT · OUT-01-SOLVE.TXT

LIMITES ET PROVENANCE

Le résultat montré provient d’une reproduction hors ligne conservée lors du backfill. Cette présentation n’a relancé aucun solveur. Les premiers journaux d’outils et l’essai initial de démodulation ne sont pas conservés.

SLIDE 32 / 46 · 60–75 secondes

Driver radio — le message et sa preuve

À DIRE

« Le défi demande d'envoyer exactement “Box Box Box!” aux 22 pilotes. Mais le programme exige aussi une signature valable pour le pilote choisi. La clé privée permet de produire cette signature ; la clé publique permet de la vérifier. Les clés publiques sont dans le programme reçu. Il manque donc le secret qui permet de signer. Une signature ne cache pas le message : elle permet ici de vérifier qu'il est autorisé. »

DÉROULÉ DES ANIMATIONS

1. Poser l'objectif : un ordre accepté par 22 pilotes.
2. → Expliquer clé privée et signature.
3. → Expliquer le contrôle avec la clé publique.
4. → Nommer le secret manquant.

POUR COMPRENDRE

Le binaire fourni est un ELF x86-64 avec symboles et informations de debug. Les fonctions pertinentes sont menu_spy, menu_send, ecdsa_sign et ecdsa_verify. menu_send vérifie la signature ECDSA P-256 et marque le pilote si le texte est exactement “Box Box Box!”. Le programme affiche le flag lorsque les 22 indicateurs sont activés. Les tableaux PILOT_QX et PILOT_QY contiennent les coordonnées des 22 clés publiques.

QUESTIONS PROBABLES

Pourquoi lire une clé publique n'est-il pas déjà une faille ? Elle est faite pour être connue. On doit pouvoir vérifier une signature sans pouvoir retrouver la clé privée. C'est une autre faiblesse qui permet ici de franchir cette séparation.

SOURCES

driver_radio/JOURNAL.md : entrées 20:00 et 20:05 ; claude/send_exploit.py.

LIMITES ET PROVENANCE

challenge.md est aussi un modèle vide dans driver_radio. Le contexte de la slide vient du journal et de la logique documentée du binaire. Le dessin de radio est une illustration pédagogique, pas une capture du programme.

SLIDE 33 / 46 · 60–75 secondes

Driver radio — un hasard trop petit

À DIRE

« La signature utilise un autre secret, temporaire : le nonce. Ce n'est pas la clé privée. Dans ce programme, il n'a que 255 valeurs possibles. Le menu Spy montre une signature, même si le message est chiffré. Une partie de cette signature dépend du nonce. L'agent peut donc essayer toutes les valeurs et comparer. Cette recherche retrouve le nonce ; elle ne donne pas encore à elle seule la clé du pilote. »

DÉROULÉ DES ANIMATIONS

1. Distinguer le nonce de la clé privée.
2. → La signature donne une valeur à comparer.
3. → Tous les candidats peuvent être essayés.
4. → Dire ce qui manque encore.

POUR COMPRENDRE

Le journal donne k = (rand() % 255) + 1 et r = (k·G).x mod n. G est le point de base de la courbe ; n est son ordre. Le solveur prépare un dictionnaire des valeurs de r associées aux 255 k possibles. Ce calcul est fait une fois, puis r sert de clé de recherche. Le message est montré chiffré en AES-CTR, mais la signature (r,s) reste visible. La faille exposée concerne la génération du nonce ECDSA.

QUESTIONS PROBABLES

Est-ce une recherche de 255 clés privées ? Non. La grille représente seulement les 255 nonces permis par cette implémentation. La clé privée reste à récupérer avec la signature et le message. La courbe P-256 ne possède évidemment pas seulement 255 clés. Le nombre gagnant n'est pas simulé sur cette slide.

SOURCES

driver_radio/JOURNAL.md : 20:05 et 20:15 ; claude/recover.py, dictionnaire rk ; screenshots/claude-02-nonce-vuln.annotated.png.

LIMITES ET PROVENANCE

Le fichier de décompilation conservé est vide. La région vulnérable est documentée dans le journal et la capture annotée ; le désassemblage complet n'a pas été gardé. Cette capture est une preuve documentaire, pas du code extrait à nouveau pour cette présentation.

Driver radio — le déclic des messages

À DIRE

« Le nonce est retrouvé, mais le texte signé reste caché. La première idée était de reproduire le hasard à partir de l'heure. Le déclic a été de regarder les messages : ils viennent de quelques modèles. Un mode moteur, un plan, un tour d'arrêt… Les variables ont peu de valeurs possibles. L'agent a donc énuméré les messages, environ 800 par pilote. Chaque message devient une hypothèse à tester. »

DÉROULÉ DES ANIMATIONS

1. Expliquer brièvement la piste de l'horloge.
2. → Lire un modèle de phrase et ses variables.
3. → Faire apparaître le nombre limité de candidats.
4. → Résumer ce que le déclic simplifie.

POUR COMPRENDRE

Le journal décrit environ 800 messages par pilote, issus de huit types de messages et de petites plages de valeurs, volontairement élargies. recover.py conserve des candidats supplémentaires pour couvrir ces plages. Le fichier permet de vérifier les formats affichés. Le nom du pilote fait partie du message : "[pilote]" sur la slide est une abréviation pédagogique. Les chaînes exactes incluent le nom complet.

QUESTIONS PROBABLES

L'agent a-t-il cassé AES ? Le solve final n'en a pas besoin : il énumère les textes plausibles et vérifie la clé privée résultante contre la clé publique connue. Pourquoi abandonner l'horloge ? La nouvelle approche élimine la dépendance à la seed, à l'heure et à l'alignement des appels de rand().

SOURCES

driver_radio/JOURNAL.md : 20:07, 20:12, 20:15 ; claude/recover.py, fonction enum().

LIMITES ET PROVENANCE

La validation locale envisagée pour la première piste a été bloquée par l'absence de libcrypto.so.3 pour x86-64. Les exemples montrés sont des modèles tirés du solveur, pas une transcription d'un échange précis avec un pilote.

SLIDE 35 / 46 · 60–75 secondes

Driver radio — calculer puis confirmer

À DIRE

« On combine trois choses : la signature observée, le nonce retrouvé, et un message possible. La relation mathématique de la signature permet de calculer une clé privée candidate. Mais on ne la croit pas sur parole. On calcule sa clé publique et on vérifie qu'elle est bien celle du pilote. Si elle ne correspond pas, on essaie un autre message. Le solveur a ainsi retrouvé et confirmé les 22 clés privées. »

DÉROULÉ DES ANIMATIONS

1. Réunir les trois informations.
2. → Calculer une candidate.
3. → Expliquer le test et ses deux issues.
4. → Annoncer les 22 confirmations.

POUR COMPRENDRE

Équation ECDSA documentée : $s = k^{-1}(z + r \cdot d) \bmod n$. Multiplier par k, soustraire z, puis multiplier par l'inverse de r donne $d = r^{-1}(s \cdot k - z) \bmod n$. $z = \text{SHA256}(\text{message})$. r^{-1} désigne l'inverse multiplicatif modulo n, pas un inverse décimal. d est un entier privé, G le point de base, Q la clé publique. Le test $d \cdot G = Q$ vérifie la candidate. Dans recover.py, il correspond à eqpt(mulG(d), Q).

QUESTIONS PROBABLES

Pourquoi faut-il deviner le message ? Sans z, le calcul de d n'est pas déterminé. Pourquoi plusieurs signatures ont-elles été récupérées ? Le journal indique dix observations par pilote, soit 220 signatures ; le solveur parcourt les signatures jusqu'à trouver une clé confirmée. Il ne faut donc pas dire que l'opération historique n'a demandé qu'une seule observation par pilote.

SOURCES

driver_radio/JOURNAL.md : 20:15–20:16 ; claude/recover.py, lignes de récupération et comparaison ; claude/crypto.py.

LIMITES ET PROVENANCE

Cette présentation lit les scripts et les résultats conservés. Elle ne recalcule ni les clés ni les signatures. Les clés privées n'ont pas besoin d'être affichées pour expliquer ou démontrer le résultat.

Driver radio — 22 ordres acceptés

À DIRE

« Avec les clés retrouvées, l'agent peut signer “Box Box Box!” pour chacun des 22 pilotes. Il vérifie ces signatures localement, puis les envoie dans une nouvelle connexion. La réponse conservée montre les 22 acceptations et le flag. La conclusion est précise : ici, un nombre temporaire trop facile à deviner et des messages prévisibles ont permis de retrouver les clés de signature. »

Clin d'œil facultatif à dire : « Imaginez la tête des mécaniciens : vingt-deux pilotes arrivent en même temps ! »

DÉROULÉ DES ANIMATIONS

1. Annoncer le résultat global.
2. → Montrer la capture conservée.
3. → Reprendre les trois grandes étapes.
4. → Donner la leçon sans généraliser à toutes les signatures.

POUR COMPRENDRE

Chaîne de fichiers historique : harvest.py → sigs.json → recover.py → priv.json → send_exploit.py → final_resp.txt. Les clés sont statiques entre connexions selon le journal ; l'agent a ainsi pu récupérer hors ligne puis signer dans une nouvelle connexion. Le flag enregistré est BA26{T0t0_1ts_C4ll3d_4_M0t0r_R4c3_N0t_S3cur3_Crypt0}.

QUESTIONS PROBABLES

Est-ce une cassure générale d'ECDSA ? Les archives montrent une exploitation de cette implémentation : nonce de 1 à 255 et messages énumérables. Ne pas généraliser le résultat à toutes les implémentations ou à tous les usages de signatures.

SOURCES

driver_radio/JOURNAL.md : 20:19–20:20 ; claude/final_resp.txt ; claude/send_exploit.py ; screenshots/claude-01-flag.annotated.png.

LIMITES ET PROVENANCE

L'annotation de la capture a été ajoutée après l'événement. Le résultat est corroboré par final_resp.txt et le journal de solve. Les timings d'animation n'indiquent pas la vitesse réelle d'exploitation.

SLIDE 37 / 46 · 45–60 secondes

Tapo — le défi

À DIRE

« Une caméra, c’est aussi un petit ordinateur. Elle conserve des programmes, des réglages et des fichiers dans sa mémoire. Voici le matériel photographié pendant le travail. Le but du challenge est de retrouver un message caché. La caméra surveillait le jardin ; pour une fois, c’est elle qu’on regarde de près. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le support identifie une Tapo C500 2.0 et un dump de 8 Mio. Le firmware est le logiciel embarqué. Un dump est une copie brute de la mémoire flash. La photographie est extraite du PowerPoint fourni, sans modification de l’original.

QUESTIONS PROBABLES

Un dump est-il une vidéo ? Non : ici, c’est une copie de la mémoire qui contient le logiciel et les données de l’appareil.

SOURCES

TAPO · POWERPOINT FOURNI, SLIDES 1–4

LIMITES ET PROVENANCE

Photographie fournie par l’utilisateur. Le dump lui-même n’est pas joint à cette demande ; aucune acquisition ou exécution réalisée pour la présentation.

Tapo — le matériel et le dump

À DIRE

« La pince et les fils relient la carte à un adaptateur, qui sert d'intermédiaire avec l'ordinateur. Le support décrit une lecture de la mémoire avec un CH347 en mode SPI et flashrom. Le résultat est un fichier de huit mébioctets. Toute la suite consiste à comprendre cette copie. Beaucoup de câbles, mais une seule pièce jointe à analyser. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

SPI est un mode de communication entre composants électroniques. L'adaptateur et le montage sont visibles dans les photographies. La lecture CH347/SPI/flashrom est rapportée par les notes du PowerPoint. Nous n'en déduisons aucun brochage et ne proposons pas de procédure matérielle à partir des photos.

QUESTIONS PROBABLES

Pourquoi travailler sur une copie ? Elle permet de relire et comparer les octets hors ligne. Cela ne dit pas si le montage était lui-même sûr ou reproductible : ce n'est pas vérifié ici.

SOURCES

TAPO · PHOTOS ET NOTES DU POWERPOINT, SLIDES 3–5

LIMITES ET PROVENANCE

Les quatre photos sont fournies par l'utilisateur. Leur ordre sur les slides est pédagogique, pas une datation du travail matériel.

Tapo — distinguer compression et chiffrement

À DIRE

« Chercher BA26 dans la mémoire est un premier réflexe raisonnable. Le support rapporte plusieurs recherches sans résultat. Il y a deux couches à distinguer. La compression range les fichiers en prenant moins de place : on doit les décompresser. Le chiffrement du flag transforme ensuite son contenu : il faut retrouver la recette et ses entrées. Un ZIP et un coffre-fort ne posent pas la même question. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les fichiers sont récupérés à partir de blocs XZ, dans un système de fichiers décrit comme SquashFS. flag.enc contient 35 octets. Les six octets affichés sont les six premiers du fichier fourni. XZ et ZIP sont des formats différents ; l’analogie concerne seulement la compression.

QUESTIONS PROBABLES

L’absence de BA26 prouve-t-elle le chiffrement ? Non. C’est l’analyse ultérieure des fichiers et de ctf_unlock qui établit les deux couches.

SOURCES

TAPO · POWERPOINT SLIDES 7–10 · FLAG.ENC · SOLVE.PY

LIMITES ET PROVENANCE

Les recherches initiales sont rapportées par le support. Aucun terminal simulé n’est présenté comme une capture réelle.

Tapo — la zone oubliée

À DIRE

« Le tournant, c'est une zone que la première extraction avait manquée. Imaginez un livre sans couverture : impossible de lire son titre, mais la table des matières et les pages existent toujours. Ici, les métadonnées indiquent encore où retrouver les fichiers. On peut donc les récupérer directement. Le livre était mal étiqueté, pas vide. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le script utilise une base à 0x1c0000, décompresse deux blocs de métadonnées XZ aux offsets 0x3a8236 et 0x3a887c, puis une table de fragments à 0x3aa18a. La fonction fragment_file lit les informations d'inode et de fragment pour extraire les données. Précision de la version (2), slide 14 : read_bytes lit le dump ; xz décompresse les blocs ; les tables décrivent les fichiers et la position de leurs données. fragment_file repère le bloc, le décompresse si nécessaire, puis découpe les octets du fichier. Les positions sont déjà connues et codées dans le script : il est spécifique à ce dump, sans découverte automatique. unlock contient le programme extrait, encrypted contient le message chiffré. Le PPT rapporte un superbloc SquashFS non reconnaissable.

QUESTIONS PROBABLES

L'en-tête a-t-il été saboté volontairement ? Le support ne le démontre pas. On peut seulement dire qu'il n'était pas reconnaissable et que des métadonnées restaient exploitables.

SOURCES

TAPO · POWERPOINT SLIDES 9–10 · SOLVE.PY

LIMITES ET PROVENANCE

L'analogie du livre est pédagogique. Les offsets sont lisibles dans le script, mais l'extraction n'a pas été relancée.

SLIDE 41 / 46 · 45–60 secondes

Tapo — les ingrédients

À DIRE

« On trouve le message chiffré, un petit programme nommé ctf_unlock, et le script de démarrage qui dit comment les relier. Ce dernier utilise l'identifiant de la caméra. Autrement dit, la caméra a fermé le coffre, mais elle a laissé la recette d'ouverture dans le même carton. Il reste à comprendre cette recette. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le fichier ctf_unlock fourni est un exécutable ELF 32 bits MIPS, statique et stripped. Le PPT indique que le script de démarrage lit devId, le transmet au programme avec flag.enc et écrit le résultat dans /tmp/flag.txt. solve.py récupère devId dans le dump via une expression régulière.

QUESTIONS PROBABLES

L'identifiant est-il une clé secrète inaccessible ? Dans ce challenge, il est récupérable dans le dump. Cette observation ne démontre pas une vulnérabilité générale de tous les produits Tapo.

SOURCES

TAPO · POWERPOINT SLIDES 11–12 · CTF_UNLOCK · FLAG.ENC · SOLVE.PY

LIMITES ET PROVENANCE

Le script de démarrage et le désassemblage sont décrits dans le PowerPoint ; ils ne sont pas fournis séparément. Le binaire n'a pas été exécuté.

Tapo — le masque

À DIRE

« Le programme fabrique une suite de nombres à partir de l'identifiant, d'un petit texte fixe et d'un compteur. Cette suite sert de masque. Le chiffrement utilise une opération appelée XOR : avec le même masque, on peut retrouver le message d'origine. L'agent a donc traduit la recette en Python. Pas de boule de cristal : les ingrédients étaient déjà dans le fichier. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Recette lisible dans solve.py : SHA256(devId_ASCII || C500-CTF-v1 || LE32(compteur)), compteur à partir de zéro. Chaque empreinte donne 32 octets ; deux empreintes couvrent les 35 octets chiffrés. Le XOR est appliqué octet par octet. Le devId est utilisé tel quel en ASCII, sans conversion hexadécimale. Précisions de la version (2), slide 15 : re.search récupère la valeur du champ devId ; struct.pack encode le compteur sur quatre octets little-endian ; digest produit 32 octets par empreinte ; join assemble les blocs. Le calcul du nombre de blocs arrondit vers le haut : 35 octets nécessitent les compteurs 0 et 1. zip associe les octets chiffrés au flux et s'arrête au plus court : seuls 35 octets servent. Le symbole ^ réalise XOR, bytes reconstitue le résultat et decode le transforme en texte. Le test assert vérifie le format BA26, pas l'acceptation par la plateforme, confirmée séparément par l'utilisateur.

QUESTIONS PROBABLES

A-t-on cassé SHA-256 ? Non. La fonction est utilisée dans le sens normal pour reconstruire un flux dont les entrées sont connues. Le script extrait ctf_unlock mais ne l'exécute pas.

SOURCES

TAPO · SOLVE.PY · POWERPOINT SLIDE 13

LIMITES ET PROVENANCE

La recette est vérifiée par lecture du script. Aucun déchiffrement ni extraction n'a été relancé pour ces slides.

SLIDE 43 / 46 · 45–60 secondes

Tapo — le XOR, sans magie

À DIRE

« Voici une opération qui tient sur quatre bits. Si les deux bits sont identiques, XOR donne zéro ; sinon, il donne un. Qui trouve le résultat de la colonne de gauche ? Laisser quelques secondes. Un, un, zéro, zéro. Maintenant, reprenons ce résultat avec le même masque : on retrouve le message de départ. C’est exactement le principe utilisé pour enlever le masque du fichier chiffré. »

DÉROULÉ DES ANIMATIONS

1. Expliquer la règle et faire calculer le public.
2. → Révéler 1100.
3. → Réappliquer le même masque : 1010.
4. → Relier le calcul au déchiffrement du fichier.

POUR COMPRENDRE

L’identité est (message XOR masque) XOR masque = message. Le script l’applique aux octets : chaque octet contient huit bits. Le masque réel vient des deux empreintes SHA-256 de la slide précédente.

QUESTIONS PROBABLES

Est-ce un chiffrement faible par nature ? XOR est une opération ; la confidentialité dépend notamment du secret et de l’usage du masque. Ici, ses ingrédients sont récupérables.

SOURCES

TAPO · POWERPOINT SLIDES 14–15 · OPÉRATION ^ DANS SOLVE.PY

LIMITES ET PROVENANCE

Mini-exemple inventé, explicitement distinct des octets réels. L’animation attend l’orateur avant de révéler la réponse.

Tapo — comprendre le script

À DIRE

« La première ligne assemble l'identifiant, l'étiquette fixe et le compteur. La deuxième calcule un bloc du masque. Deux blocs suffisent ici. La troisième associe chaque octet du message chiffré à celui du masque, puis fait le XOR. Ce code reprend la logique du programme MIPS ; il n'exécute pas ce programme sur notre ordinateur. »

DÉROULÉ DES ANIMATIONS

1. Relier key à devId et expliquer le compteur.
2. → SHA-256 fabrique le masque.
3. → XOR retrouve les octets du texte.
4. → Distinguer recette, résultat conservé et soumission.

POUR COMPRENDRE

Les trois expressions sont issues du solve.py fourni, avec les arguments de sha256 abrégés par une ellipse. struct.pack encode un entier non signé sur quatre octets little-endian. digest renvoie 32 octets. Le nombre de blocs est (len(encrypted)+31)//32. zip s'arrête après les 35 octets chiffrés. En amont, fragment_file extrait ctf_unlock et flag.enc à partir des tables décompressées ; re.search retrouve devId. Les offsets sont fixes : ce solveur est spécifique au dump. Le test assert contrôle le format BA26{…}, pas une validation serveur.

QUESTIONS PROBABLES

Pourquoi le programme est-il extrait s'il n'est pas lancé ? Il documente la recette. Le solveur reproduit ensuite cette logique en Python.

SOURCES

TAPO · SOLVE.PY · POWERPOINT SLIDES 17–18

LIMITES ET PROVENANCE

Lecture statique du script fourni. Aucun déchiffrement, téléchargement ni exécution du binaire effectué pour cette présentation.

SLIDE 45 / 46 · 45–60 secondes

Tapo — le résultat

À DIRE

« Le fichier flag.txt fourni contient ce texte, exactement identique à celui du PowerPoint. On dispose donc du résultat et d'un script qui décrit une procédure pour le retrouver depuis le dump. La soumission a été confirmée par la plateforme : le challenge est validé. La caméra avait gardé son secret et son mode d'emploi au même endroit. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les fichiers reçus sont Resolution_CTF_Tapo.pptx, solve.py, ctf_unlock, flag.enc et flag.txt. flag.enc et flag.txt font chacun 35 octets. Le flag lu correspond au support. Le script vérifie la forme BA26{…}, mais cette vérification n'est pas une validation serveur.

QUESTIONS PROBABLES

Avons-nous reproduit le solve ici ? Non : les fichiers ont été lus, et le script inspecté. Le dump n'a pas été fourni dans cette demande, et aucune exécution n'a été réalisée. La reproduction annoncée est celle du PowerPoint source.

SOURCES

TAPO · FLAG.TXT FOURNI · SOUMISSION CONFIRMÉE PAR L'UTILISATEUR

LIMITES ET PROVENANCE

La concordance entre flag.txt et le PowerPoint est vérifiée. L'utilisateur a confirmé la soumission à la plateforme après la réception du PowerPoint ; cette confirmation remplace le statut provisoire du support. La provenance de la résolution et son auteur ne sont pas précisés ; aucune attribution à un agent n'est inventée.

Conclusion

À DIRE

« Les attaques sont différentes, mais on peut les raconter avec les mêmes questions. Quel était le verrou ? Quelle hypothèse était fragile ? Et quelle preuve nous permet de dire que cela a fonctionné ? C’est ce passage du résultat au raisonnement qui nous permet de nous approprier le travail des agents. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les schémas de cette présentation simplifient les mécanismes. Les notes rattachent chaque affirmation aux journaux et aux artefacts disponibles. Les captures annotées produites après l’événement sont identifiées comme telles.

QUESTIONS PROBABLES

Les chapitres peuvent être présentés séparément. Pour un créneau court, commencer par les embeddings, id_please et wrong_path. Les mécanismes les plus spécialisés restent détaillés dans leurs notes.

SOURCES

JOURNAUX ET ARTEFACTS CONSERVÉS · PRÉSENTATION DOCUMENTAIRE

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.